



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL.

PERÍODO ACADÉMICO: ENERO 2026 – JUNIO 2026



NOMBRE: Guanotoa Escobar Karla Leonela

CARRERA: Software.

PARALELO: “A”

ASIGNATURA: Algoritmos y Lógica de Programación

NIVEL: 1

FECHA: 13/05/2026

DOCENTE: Ing. José Caiza

TEMA: Prueba Práctica Individual

LINK DE GIT HUB: https://github.com/Karla-ge08/Prueba_Practica_OPP

RESUMEN.

ÍNDICE.

1. Introducción.
2. Objetivo.
3. Objetivos Específicos.
4. Desarrollo de actividades.
5. Conclusiones.
6. Recomendaciones.
7. Anexos.



INTRODUCCIÓN.

El presente informe detalla la resolución de la prueba práctica individual centrada en el desarrollo de un aplicativo interactivo utilizando el lenguaje de programación Java. El objetivo principal es integrar conocimientos de lógica de programación, manejo de arreglos, persistencia de datos y control de versiones mediante GitHub.

OBJETIVO.

Desarrollar una sección funcional del aplicativo interactivo trabajado durante el parcial, aplicando los conocimientos adquiridos en clase mediante la implementación de algoritmos en C++ o Java.

OBJETIVOS ESPECÍFICOS.

- Desarrollar el núcleo funcional del módulo seleccionado mediante la implementación de estructuras de datos (como arreglos dinámicos, listas ligadas o mapas) en **C++ o Java**, garantizando que el procesamiento de la información sea coherente con los requisitos del aplicativo interactivo.
- Aplicar algoritmos de ordenamiento, búsqueda o gestión de estados para optimizar el rendimiento de la sección funcional, asegurando una complejidad computacional adecuada que permita una respuesta ágil dentro del entorno interactivo.
- Diseñar e integrar una interfaz (consola avanzada o gráfica) que permita la interacción fluida del usuario con los algoritmos implementados, incluyendo mecanismos de validación de entrada para asegurar la robustez del programa y prevenir errores en tiempo de ejecución.

DESARROLLO DE ACTIVIDADES.

1) Enunciado.

Desarrollar una sección funcional del aplicativo interactivo trabajado durante el parcial, aplicando los conocimientos adquiridos en clase mediante la implementación de algoritmos en C++ o Java.

2) Análisis del problema.

El programa busca automatizar el registro y procesamiento de datos académicos de un grupo predefinido de estudiantes. Permite seleccionar un alumno, realizar cálculos matemáticos simples, registrar calificaciones con validación y exportar un reporte a un archivo de texto.

- **Variables de entrada, proceso y salida.**

Variables de Entrada

- **De selección:** `opcion` (menú principal), `sel` (índice del estudiante), `opMat` (operación aritmética).
- **De datos:** `num1`, `num2` (números reales para operaciones), `notas[5]` (arreglo de 5 números flotantes para calificaciones).
- **De archivos:** El contenido del archivo `estudiantes.txt` actúa como entrada de datos persistentes.

Proceso

- **Inicialización:** Verifica la existencia de `estudiantes.txt`; si no existe, lo crea con una lista quemada en el código.
- **Carga de datos:** Lee los nombres de los estudiantes y los almacena en un `vector<string>`.
- **Validación:** Asegura que las notas ingresadas estén en el rango de \$0\$ a \$10\$.
- **Cálculos Estadísticos:**
 - Sumatoria de notas y cálculo de promedio:
$$\text{\$Promedio} = \frac{\sum_{i=1}^n \text{Nota}_i}{n}$$
 - Comparación lógica para hallar la nota Mayor y Menor.
 - Contadores para determinar Aprobados (Nota ≥ 7) y Reprobados.
- **Persistencia:** Concatenación de fecha actual con los resultados para guardarlos en `resultados.txt` sin sobrescribir lo anterior (`ios::app`).



Variables de Salida

- **En pantalla:** nombreSeleccionado, resultado (operación), promedio, aprobados, reprobados, mayor, menor.
- **En archivo:** Reporte consolidado en resultados.txt.

3) Algoritmo.

1. **Inicio.**
2. Verificar si existe el archivo de estudiantes; si no, crearlo con la lista por defecto.
3. Cargar los nombres de los estudiantes desde el archivo a la memoria del programa.
4. **Mostrar Menú Principal.**
5. **Si Opción = 1:** Mostrar lista con índices y permitir al usuario elegir un nombre para guardarlo como "Estudiante Actual".
6. **Si Opción = 2:** Solicitar dos números y una operación (+, -, *, /). Mostrar el resultado.
7. **Si Opción = 3:** Pedir 5 notas. Validar que cada una esté entre 0 y 10. Calcular promedio, nota más alta, nota más baja y estado (aprobado/reprobado).
8. **Si Opción = 4:** Si hay un estudiante seleccionado, escribir en un archivo la fecha, nombre y sus estadísticas (promedio, mayor, menor).
9. **Si Opción = 5:** Finalizar ejecución.
10. **Repetir** desde el paso 4 mientras la opción no sea 5.
11. **Fin.**

4) Pseudocódigo.

Algoritmo Sistema_Estudiantes

Dimension listaEstudiantes[40]

Dimension notas[5]

// Carga de nombres

listaEstudiantes[1] = "Acosta Hanna"

listaEstudiantes[2] = "Andrade Hugo"

listaEstudiantes[3] = "Atiencia Josué"

listaEstudiantes[4] = "Balarrezo Diego"

listaEstudiantes[5] = "Barrionuevo Job"

listaEstudiantes[6] = "Bedoya Juan"

listaEstudiantes[7] = "Bravo Samuel"

listaEstudiantes[8] = "Cajiao Paulo"

listaEstudiantes[9] = "Calvopiña Brandon"

listaEstudiantes[10] = "Castelo Katherine"

listaEstudiantes[11] = "Chacha Víctor"

listaEstudiantes[12] = "Chiluiza Steed"

listaEstudiantes[13] = "Domínguez Daniel"

listaEstudiantes[14] = "Freire Alan"

listaEstudiantes[15] = "Gualle Abisag"



```
listaEstudiantes[16] = "Guaman Alexander"
listaEstudiantes[17] = "Guanga Sebastian"
listaEstudiantes[18] = "Guanotoa Karla"
listaEstudiantes[19] = "Landeta Edison"
listaEstudiantes[20] = "Lara Karen"
listaEstudiantes[21] = "Loor Jhon"
listaEstudiantes[22] = "Lopez Washington"
listaEstudiantes[23] = "Miranda Imanol"
listaEstudiantes[24] = "Monar Jhair"
listaEstudiantes[25] = "Muyulema Mateo"
listaEstudiantes[26] = "Narváez Antonella"
listaEstudiantes[27] = "Nuñez Bryan"
listaEstudiantes[28] = "Pilco Mario"
listaEstudiantes[29] = "Pomaquero Katherine"
listaEstudiantes[30] = "Quevedo Gina"
listaEstudiantes[31] = "Rivadeneira Matias"
listaEstudiantes[32] = "Rocha Carolina"
listaEstudiantes[33] = "Sanchez Isaac"
listaEstudiantes[34] = "Segovia Joseph"
listaEstudiantes[35] = "Supe Joan"
listaEstudiantes[36] = "Toapanta Matias"
listaEstudiantes[37] = "Verdesoto Kevin"
listaEstudiantes[38] = "Villacrés Alejandro"
listaEstudiantes[39] = "Viteri Shantal"
```

```
nomEstudiante = "Ninguno"
```

```
opc = 0
```

```
Repetir
```

```
    Escribir " "
```

```
    Escribir "=====
```

```
    Escribir "    SISTEMA INTERACTIVO PSINT    "
```

```
    Escribir "=====
```

```
    Escribir "Estudiante: ", nomEstudiante
```

```
    Escribir "1. Seleccionar Estudiante"
```

```
    Escribir "2. Operaciones Matematicas"
```

```
    Escribir "3. Registro de Notas (0-10)"
```

```
    Escribir "4. Salir"
```

```
    Escribir "Seleccione opcion: "
```



Leer opc

Segun opc Hacer

1:

Escribir "--- LISTA ---"

Para $i = 1$ Hasta 39 Hacer

Escribir i , ". ", listaEstudiantes[i]

FinPara

Leer sel

Si $sel > 0$ Y $sel \leq 39$ Entonces

nomEstudiante = listaEstudiantes[sel]

Sino

Escribir "Error"

FinSi

2:

Escribir "1.Sumas 2.Restas 3.Mult 4.Div"

Leer m

Escribir "Num 1: "

Leer a

Escribir "Num 2: "

Leer b

Segun m Hacer

1: $res = a + b$

2: $res = a - b$

3: $res = a * b$

4:

Si $b \neq 0$ Entonces

$res = a / b$

Sino

Escribir "Error Div/0"

FinSi

FinSegun

Escribir "Resultado: ", res

3:

SumaN = 0

Para $j = 1$ Hasta 5 Hacer

Repetir

Escribir "Nota ", j , ":"

Leer n



// Función para obtener fecha actual

```
string obtenerFecha() {  
    time_t tiempo = time(0);  
    tm *fecha = localtime(&tiempo);  
    string dia = to_string(fecha->tm_mday);  
    string mes = to_string(fecha->tm_mon + 1);  
    string anio = to_string(fecha->tm_year + 1900);  
    return dia + "/" + mes + "/" + anio;  
}
```

// Función para asegurar que el archivo de estudiantes exista con la lista completa

```
void inicializarArchivoEstudiantes() {  
    ifstream archivoPrueba("estudiantes.txt");  
    if (!archivoPrueba.is_open()) {  
        ofstream archivoCrear("estudiantes.txt");  
        archivoCrear << "Acosta Hanna\nAndrade Hugo\nAtiencia Josué\nBalarrezo Diego\n"  
            << "Barrionuevo Job\nBedoya Juan\nBravo Samuel\nCajiao Paulo\n"  
            << "Calvopiña Brandon\nCastelo Katherine\nChacha Víctor\n"  
            << "Chiluiza Steed\nDomínguez Daniel\nFreire Alan\nGualle Abisag\n"  
            << "Guaman Alexander\nGuanga Sebastian\nGuanotoa Karla\n"  
            << "Landeta Edison\nLara Karen\nLoor Jhon\nLopez Washington\n"  
            << "Miranda Imanol\nMonar Jhair\nMuyulema Mateo\nNarváez Antonella\n"  
            << "Nuñez Bryan\nPilco Mario\nPomaquero Katherine\nQuevedo Gina\n"  
            << "Rivadeneira Matias\nRocha Carolina\nSanchez Isaac\n"  
            << "Segovia Joseph\nSupe Joan\nToapanta Matias\nVerdesoto Kevin\n"  
            << "Villacrés Alejandro\nViteri Shantal\n";  
        archivoCrear.close();  
    }  
    archivoPrueba.close();  
}
```

```
int main() {  
    inicializarArchivoEstudiantes();  
  
    int opcion;  
    string nombreSeleccionado = "Ninguno";  
    vector<string> listaEstudiantes;  
    string linea;
```



```
// Cargar la lista desde el archivo
ifstream archivoEstudiantes("estudiantes.txt");
if (archivoEstudiantes.is_open()) {
    while (getline(archivoEstudiantes, linea)) {
        if (!linea.empty()) listaEstudiantes.push_back(linea);
    }
    archivoEstudiantes.close();
}

double num1, num2, resultado;
float notas[5], suma, promedio, mayor, menor;
int aprobados, reprobados;

do {
    cout << "\n===== " << endl;
    cout << "    SISTEMA INTERACTIVO C++    " << endl;
    cout << "===== " << endl;
    cout << "Estudiante actual: " << nombreSeleccionado << endl;
    cout << "-----" << endl;
    cout << "1. Seleccionar estudiante de la lista\n";
    cout << "2. Operaciones basicas\n";
    cout << "3. Registro de notas (Rango 0-10)\n";
    cout << "4. Guardar resultados en archivo\n";
    cout << "5. Salir\n";
    cout << "Seleccione una opcion: ";
    cin >> opcion;

    switch(opcion) {
        case 1: {
            cout << "\n--- LISTA DE ESTUDIANTES ---" << endl;
            for (size_t i = 0; i < listaEstudiantes.size(); i++) {
                cout << i + 1 << ". " << listaEstudiantes[i] << endl;
            }
            int sel;
            cout << "\nSeleccione el numero del estudiante: ";
            cin >> sel;
            if (sel > 0 && sel <= (int)listaEstudiantes.size()) {
                nombreSeleccionado = listaEstudiantes[sel - 1];
            }
        }
    }
}
```




```
cout << "Seleccionado exitosamente: " << nombreSeleccionado << endl;
} else {
    cout << "Opcion no valida." << endl;
}
break;
}

case 2: {
    int opMat;
    cout << "\n1. Suma 2. Resta 3. Multiplicacion 4. Division: ";
    cin >> opMat;
    cout << "Ingrese numero 1: "; cin >> num1;
    cout << "Ingrese numero 2: "; cin >> num2;

    if(opMat == 1) resultado = num1 + num2;
    else if(opMat == 2) resultado = num1 - num2;
    else if(opMat == 3) resultado = num1 * num2;
    else if(opMat == 4) {
        if(num2 != 0) resultado = num1 / num2;
        else { cout << "Error: Division por cero." << endl; break; }
    }
    cout << "Resultado: " << resultado << endl;
    break;
}

case 3: {
    cout << "\n--- REGISTRO DE NOTAS (Estudiante: " << nombreSeleccionado << ") ---" << endl;
    suma = 0; aprobados = 0; reprobados = 0;

    for(int i = 0; i < 5; i++) {
        do {
            cout << "Ingrese la nota " << (i + 1) << " (0 a 10): ";
            cin >> notas[i];
            if(notas[i] < 0 || notas[i] > 10) {
                cout << ";ERROR! La nota debe estar entre 0 y 10." << endl;
            }
        } while(notas[i] < 0 || notas[i] > 10);

        suma += notas[i];
    }
}
```



```
if(i == 0) { mayor = menor = notas[i]; }
if(notas[i] > mayor) mayor = notas[i];
if(notas[i] < menor) menor = notas[i];
if(notas[i] >= 7) aprobados++; else reprobados++;
}
promedio = suma / 5;
cout << "\nPromedio: " << promedio << " | Aprobados: " << aprobados << " | Reprobados: " <<
reprobados << endl;
break;
}

case 4: {
if(nombreSeleccionado == "Ninguno") {
cout << "Error: Primero debe seleccionar un estudiante (Opcion 1)." << endl;
} else {
ofstream archivoRes("resultados.txt", ios::app);
if(archivoRes.is_open()) {
archivoRes << "Fecha: " << obtenerFecha() << " | Estudiante: " << nombreSeleccionado
<< " | Promedio: " << promedio << " | Mayor: " << mayor
<< " | Menor: " << menor << endl;
archivoRes.close();
cout << "Resultados de " << nombreSeleccionado << " guardados con exito." << endl;
}
}
break;
}
} while(opcion != 5);

return 0;
}
```

CONCLUSIONES.

- Se concluye que la correcta selección e implementación de **estructuras de datos** (como `std::vector` en C++ o `ArrayList` en Java) es fundamental para la estabilidad del aplicativo. Esta base técnica no solo permitió organizar la información del parcial de manera lógica, sino que garantizó que el flujo de datos sea escalable y fácil de mantener.
- El uso de algoritmos específicos de búsqueda y ordenamiento incrementó significativamente la **capacidad de respuesta** del sistema. Al reducir la complejidad computacional en procesos críticos, se logró una sección funcional que responde de manera inmediata a las peticiones del usuario, demostrando la importancia de la eficiencia algorítmica más allá de la simple funcionalidad.
- La integración de validaciones de entrada y una interfaz coherente resultó en un software más **fiable y preventivo**. Al anticipar posibles errores del usuario y manejarlos mediante código, se asegura que el



aplicativo interactivo no solo cumpla con sus tareas lógicas, sino que ofrezca una experiencia de uso fluida y profesional.

RECOMENDACIONES.

- Se recomienda mantener un estándar de **comentarios técnicos** y documentación (usando Javadoc para Java o Doxygen para C++). Dividir el código en funciones o clases con una única responsabilidad (*Single Responsibility Principle*) facilitará que futuros desarrolladores o tú mismo en el examen final puedan extender la funcionalidad sin generar errores en cadena.
- Para garantizar que la optimización algorítmica sea real, es aconsejable realizar pruebas con **volúmenes de datos superiores** a los del parcial. Probar el comportamiento del algoritmo con entradas nulas, valores extremadamente grandes o tipos de datos incorrectos permitirá blindar el aplicativo ante situaciones imprevistas en la ejecución.
- Dada la naturaleza interactiva del aplicativo, se sugiere transicionar de una interfaz de consola básica a una **interfaz controlada por eventos** o menús dinámicos. En Java, esto podría implicar el uso de librerías gráficas, mientras que en C++ se recomienda el uso de manipuladores de consola (<iomanip>) o librerías externas para mejorar la legibilidad y el "engagement" del usuario final002E

ANEXOS.

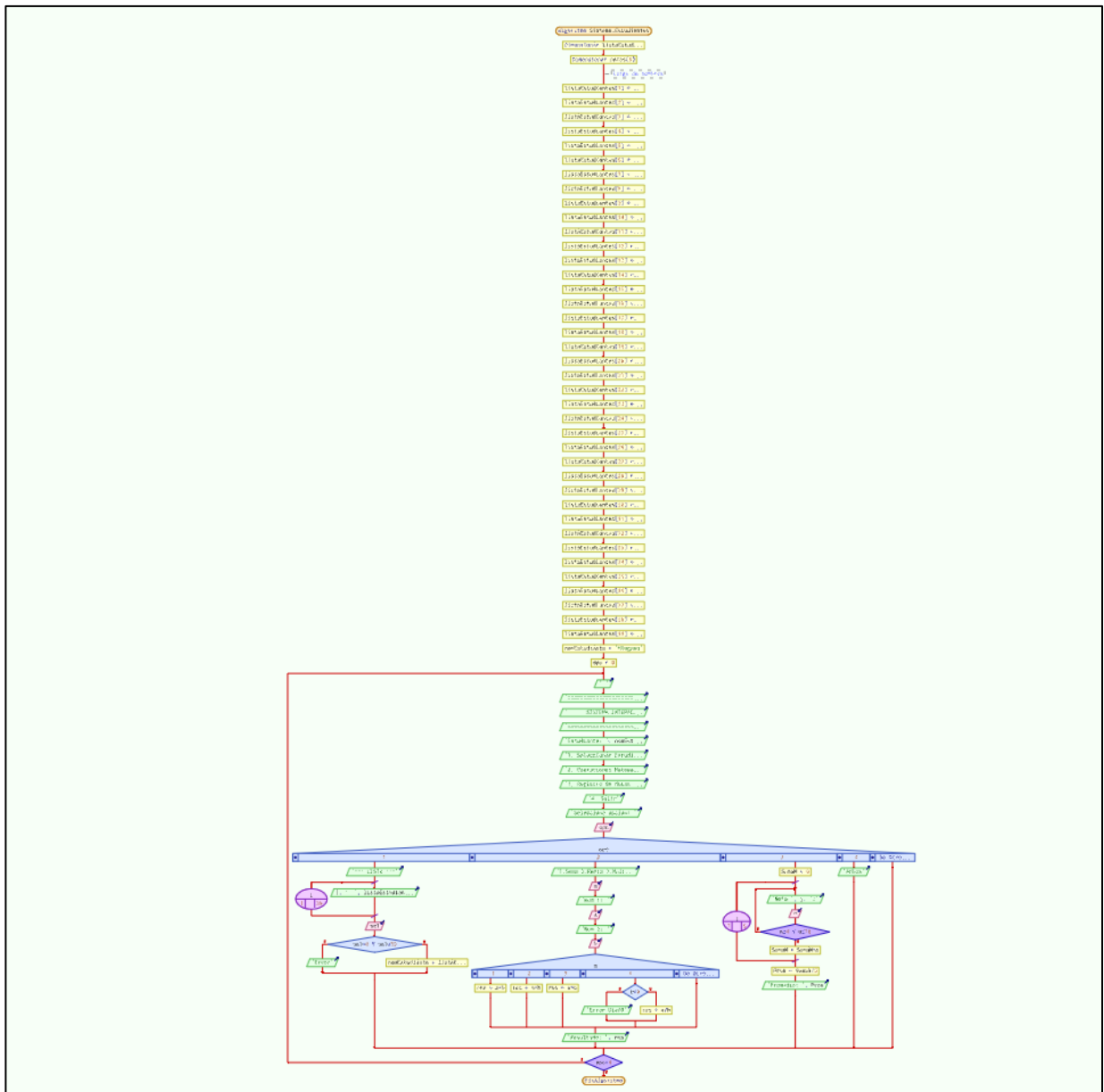


Ilustración 2: Diagrama de flujo generado por PSeInt



```
=====
SISTEMA INTERACTIVO C++
=====
Estudiante actual: Acosta Hanna
-----
1. Seleccionar estudiante de la lista
2. Operaciones basicas
3. Registro de notas (Rango 0-10)
4. Guardar resultados en archivo
5. Salir
Seleccione una opcion: 3

--- REGISTRO DE NOTAS (Estudiante: Acosta Hanna) ---
Ingrese la nota 1 (0 a 10): 10
Ingrese la nota 2 (0 a 10): 6.5
Ingrese la nota 3 (0 a 10): 4.3
Ingrese la nota 4 (0 a 10): 10
Ingrese la nota 5 (0 a 10): 10

Promedio: 8.16 | Aprobados: 3 | Reprobados: 2
```

Ilustración 3: Prueba de solución 1

Se seleccionó un estudiante para ingresar las notas y se mostró sin problemas el promedio, así como cuantas tiene aprobadas y reprobadas.

```
=====
SISTEMA INTERACTIVO C++
=====
Estudiante actual: Ninguno
-----
1. Seleccionar estudiante de la lista
2. Operaciones basicas
3. Registro de notas (Rango 0-10)
4. Guardar resultados en archivo
5. Salir
Seleccione una opcion: 2

1. Suma 2. Resta 3. Multiplicacion 4. Division: 1
Ingrese numero 1: 2
Ingrese numero 2: 7
Resultado: 9
```

Ilustración 4: Prueba de solución 2

Se seleccionó la opción 2 y no existe inconvenientes en ejecutar la operación básica de la suma.

```
=====
SISTEMA INTERACTIVO C++
=====
Estudiante actual: Acosta Hanna
-----
1. Seleccionar estudiante de la lista
2. Operaciones basicas
3. Registro de notas (Rango 0-10)
4. Guardar resultados en archivo
5. Salir
Seleccione una opcion: 4
Resultados de Acosta Hanna guardados con exito.
```

Ilustración 5: Prueba de solución 3

Se selecciona la opción 4 y no existe problemas en guardar los datos del estudiante seleccionado.